



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования

«Дальневосточный федеральный университет»
(ДВФУ)

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ

Кафедра математического и компьютерного моделирования

Избосарова Жасмина Олимжоновна

**ЛАБОРАТОРНАЯ РАБОТА №3: ЭМПИРИЧЕСКАЯ ФУНКЦИЯ
РАСПРЕДЕЛЕНИЯ**

Направление подготовки 02.03.01 СЦТ «Сквозные цифровые технологии»,
образовательная программа «Математика и компьютерные науки»,
очная форма обучения

Студент группы Б9123-02.03.01сцт2

_____ Ж. О. Избосарова

Руководитель проекта

_____ К. А. Ганжа

Владивосток

2026

1 Краткое математическое описание методов и синтаксис в Python

В работе рассматриваются те же 8 выборок, что и в лабораторной работе №1: по две выборки объёмов 100 и 1000 из распределений

$$U(2, 7), \quad B(0.27), \quad \text{Bin}(12, 0.35), \quad N(4.5, 1.8^2).$$

Для выборки $X_1, \dots, X_n$ эмпирическая функция распределения определяется формулой

$$F_n(x) = \frac{1}{n} \sum_{i=1}^n I(X_i \leq x),$$

где $I(X_i \leq x)$ — индикатор того, что наблюдение не превосходит точку x .

Доверительный интервал $L(x), U(x)$ для $F(x)$ с уровнем доверия $1 - \alpha$ в нашем случае имеет вид:

$$\mathbb{P}(L(x) \leq F(x) \leq U(x)) \geq 1 - \alpha.$$

Где:

$$L(x) = \max(\hat{F}_n(x) - \varepsilon_n; 0),$$

$$U(x) = \min(\hat{F}_n(x) + \varepsilon_n; 1),$$

$$\varepsilon_n = \sqrt{\frac{1}{2n} \ln \left(\frac{2}{\alpha} \right)}.$$

В работе используется 95%-й доверительный интервал, то есть

$$\alpha = 0.05.$$

В программе нижняя и верхняя границы дополнительно ограничиваются интервалом $[0, 1]$, так как функция распределения не может принимать значения вне этого отрезка.

Во всех экспериментах фиксируется

```
1 random_state = 16
```

Генерация выборок выполняется так же, как и в ЛР №1.

Равномерное распределение: $X \sim U(2, 7)$.

Код:

```
1 st.uniform.rvs(loc=2, scale=5, size=100, random_state=16)
2 st.uniform.rvs(loc=2, scale=5, size=1000, random_state=16)
```

Распределение Бернулли: $X \sim B(0.27)$.

Код:

```
1 st.bernoulli.rvs(p=0.27, size=100, random_state=16)
2 st.bernoulli.rvs(p=0.27, size=1000, random_state=16)
```

Биномиальное распределение: $X \sim Bin(12, 0.35)$.

Код:

```
1 st.binom.rvs(n=12, p=0.35, size=100, random_state=16)
2 st.binom.rvs(n=12, p=0.35, size=1000, random_state=16)
```

Нормальное распределение: $X \sim N(4.5, 1.8^2)$.

Код:

```
1 st.norm.rvs(loc=4.5, scale=1.8, size=100, random_state=16)
2 st.norm.rvs(loc=4.5, scale=1.8, size=1000, random_state=16)
```

Собственная функция вычисления ЭФР и доверительного интервала имеет вид:

```
1 def empirical_cdf(sample, x, alpha=0.05):
2     sample = np.sort(np.asarray(sample))
3     n = sample.size
4     x_array = np.atleast_1d(x)
5
6     fn = np.searchsorted(sample, x_array, side="right") / n
7     epsilon = np.sqrt(np.log(2 / alpha) / (2 * n))
8     lower = np.clip(fn - epsilon, 0.0, 1.0)
9     upper = np.clip(fn + epsilon, 0.0, 1.0)
10
11     if np.ndim(x) == 0:
12         return float(fn[0]), float(lower[0]), float(upper[0])
13     return fn, lower, upper
```

Для сравнения используется встроенная функция из SciPy:

Код:

```

1 ecdf_result = st.ecdf(sample)
2 ecdf_values = ecdf_result.cdf.evaluate(x_grid)
3 ci = ecdf_result.cdf.confidence_interval(confidence_level
    =0.95)
4 low = ci.low.evaluate(x_grid)
5 high = ci.high.evaluate(x_grid)

```

Для непрерывных распределений графики строятся на плотной равномерной сетке по x , а для дискретных — по целочисленным значениям из области возможных исходов.

2 Результаты

2.1 Таблица результатов

В таблице приведены значения ЭФР в точке медианы каждой выборки, а также нижняя и верхняя границы 95%-го доверительного интервала.

Таблица 1: Значения ЭФР в точке медианы выборки и 95%-й доверительный интервал

Выборка	x	$F_n(x)$	Нижняя граница	Верхняя граница
uniform_100	4.6254	0.5000	0.3642	0.6358
uniform_1000	4.4807	0.5000	0.4571	0.5429
bernoulli_100	0.0000	0.7500	0.6142	0.8858
bernoulli_1000	0.0000	0.7500	0.7071	0.7929
binom_100	4.0000	0.5800	0.4442	0.7158
binom_1000	4.0000	0.5950	0.5521	0.6379
norm_100	4.6516	0.5000	0.3642	0.6358
norm_1000	4.4144	0.5000	0.4571	0.5429

По таблице видно, что при увеличении объёма выборки доверительный интервал становится уже, а значение ЭФР стабилизируется.

2.2 Графики

На графиках:

- красная линия — истинная функция распределения;

- синяя ступенчатая линия — эмпирическая функция распределения;
- зелёные пунктирные линии — нижняя и верхняя границы 95%-го доверительного интервала.

На первом рисунке приведены результаты для собственной реализации ЭФР и доверительного интервала, построенного по приведённой выше формуле.

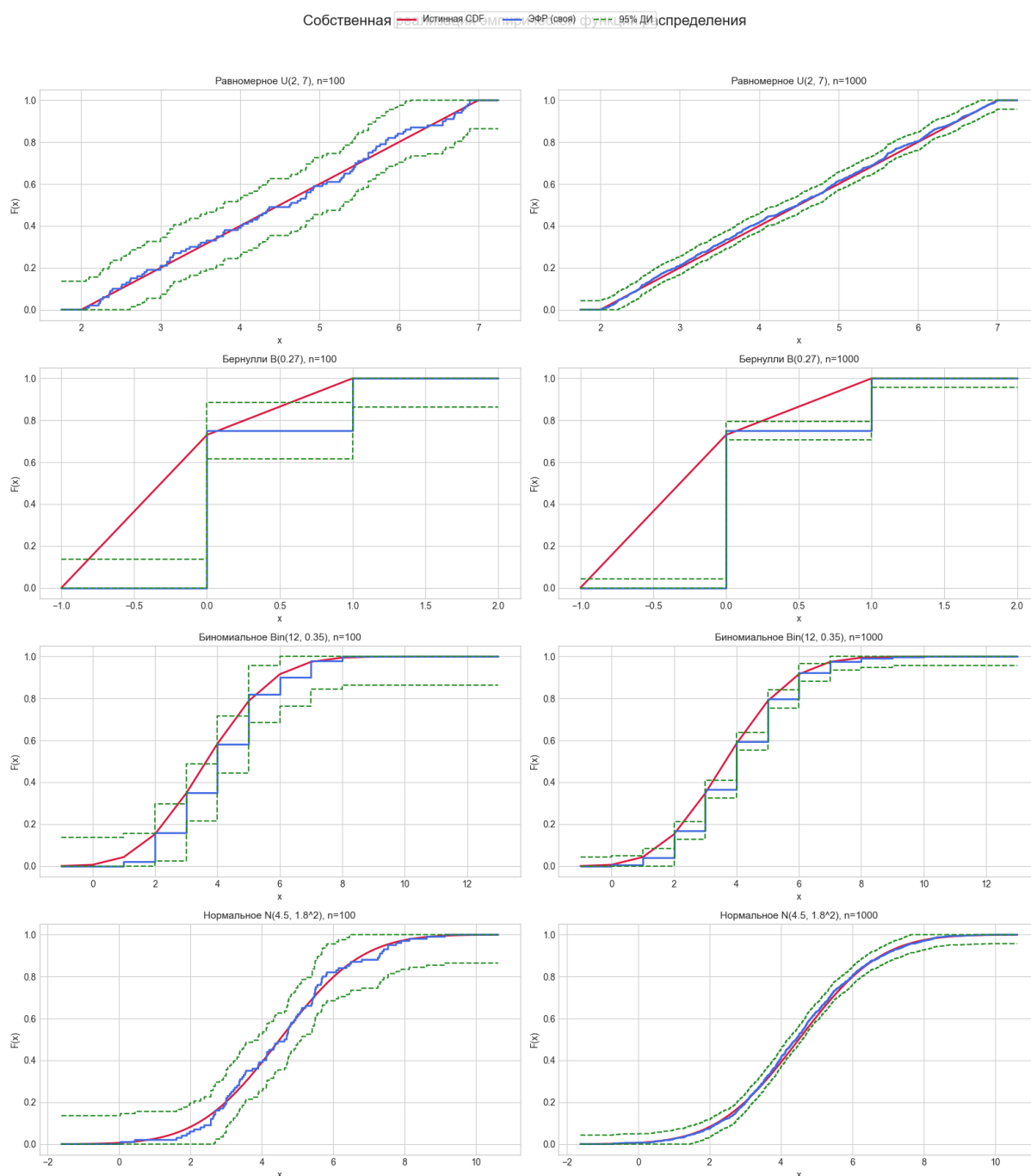


Рис. 1: Собственная реализация ЭФР и 95%-го доверительного интервала

На следующем рисунке приведены результаты, полученные с помощью `scipy.stats.ecdf`. Значения ЭФР совпадают с собственной реализацией, а доверительные интервалы могут немного отличаться, так как в SciPy используется другой способ их построения.

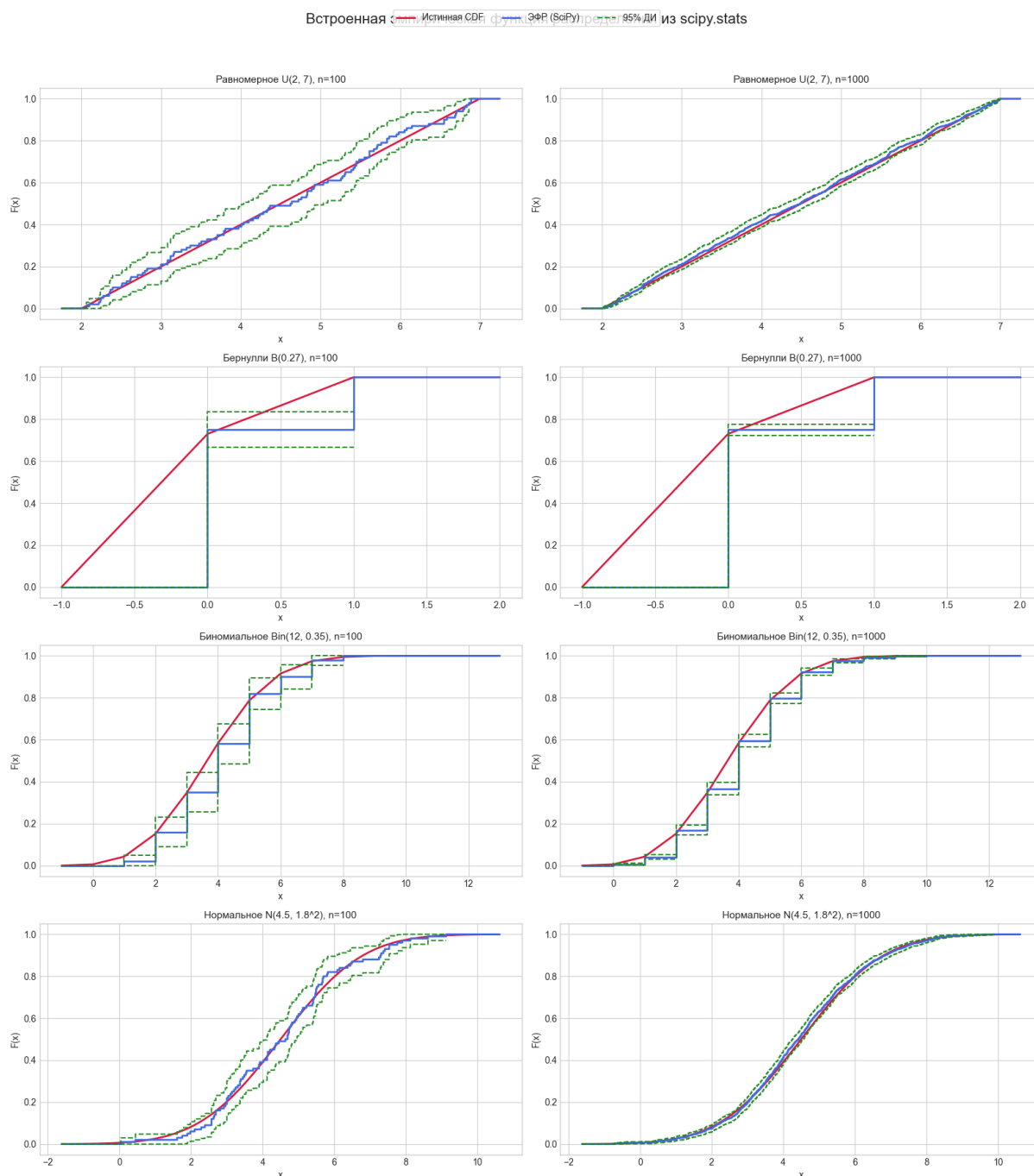


Рис. 2: ЭФР и 95%-й доверительный интервал с использованием `scipy.stats.ecdf`

2.3 Вывод

В лабораторной работе была реализована собственная функция вычисления эмпирической функции распределения и 95%-го доверительного интервала. Для всех восьми выборок построены графики истинной функции распределения, собственной ЭФР и встроенной ЭФР из SciPy.

С увеличением объёма выборки от 100 до 1000 эмпирическая функция распределения становится ближе к теоретической функции распределения, а доверительный интервал сужается. Это подтверждает корректность реализации и ожидаемое улучшение качества оценки при росте числа наблюдений.

3 Полный код программы

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import scipy.stats as st
5
6
7 plt.style.use("seaborn-v0_8-whitegrid")
8 plt.rcParams["figure.figsize"] = (16, 10)
9 plt.rcParams["axes.titlesize"] = 11
10 plt.rcParams["axes.labelsize"] = 10
11
12
13 random_state = 16
14
15
16 samples = {
17     "uniform_100": st.uniform.rvs(loc=2, scale=5, size=100,
18     random_state=random_state),
19     "uniform_1000": st.uniform.rvs(
20         loc=2, scale=5, size=1000, random_state=random_state
21     ),
22     "bernoulli_100": st.bernoulli.rvs(p=0.27, size=100,
23     random_state=random_state),
```

```

22     "bernoulli_1000": st.bernoulli.rvs(
23         p=0.27, size=1000, random_state=random_state
24     ),
25     "binom_100": st.binom.rvs(n=12, p=0.35, size=100,
random_state=random_state),
26     "binom_1000": st.binom.rvs(
27         n=12, p=0.35, size=1000, random_state=random_state
28     ),
29     "norm_100": st.norm.rvs(loc=4.5, scale=1.8, size=100,
random_state=random_state),
30     "norm_1000": st.norm.rvs(
31         loc=4.5, scale=1.8, size=1000, random_state=
random_state
32     ),
33 }
34
35
36 plot_config = [
37     {
38         "sample_name": "uniform_100",
39         "title": "Равномерное U(2, 7), n=100",
40         "kind": "continuous",
41         "dist": st.uniform(loc=2, scale=5),
42     },
43     {
44         "sample_name": "uniform_1000",
45         "title": "Равномерное U(2, 7), n=1000",
46         "kind": "continuous",
47         "dist": st.uniform(loc=2, scale=5),
48     },
49     {
50         "sample_name": "bernoulli_100",
51         "title": "Бернулли B(0.27), n=100",
52         "kind": "discrete",
53         "dist": st.bernoulli(p=0.27),
54     },
55     {
56         "sample_name": "bernoulli_1000",
57         "title": "Бернулли B(0.27), n=1000",
58         "kind": "discrete",

```



```

59     "dist": st.bernoulli(p=0.27),
60 },
61 {
62     "sample_name": "binom_100",
63     "title": "Биномиальное Bin(12, 0.35), n=100",
64     "kind": "discrete",
65     "dist": st.binom(n=12, p=0.35),
66 },
67 {
68     "sample_name": "binom_1000",
69     "title": "Биномиальное Bin(12, 0.35), n=1000",
70     "kind": "discrete",
71     "dist": st.binom(n=12, p=0.35),
72 },
73 {
74     "sample_name": "norm_100",
75     "title": "Нормальное N(4.5, 1.8^2), n=100",
76     "kind": "continuous",
77     "dist": st.norm(loc=4.5, scale=1.8),
78 },
79 {
80     "sample_name": "norm_1000",
81     "title": "Нормальное N(4.5, 1.8^2), n=1000",
82     "kind": "continuous",
83     "dist": st.norm(loc=4.5, scale=1.8),
84 },
85 ]
86
87
88 def empirical_cdf(sample, x, alpha=0.05):
89     sample = np.sort(np.asarray(sample))
90     n = sample.size
91     x_array = np.atleast_1d(x)
92
93     fn = np.searchsorted(sample, x_array, side="right") / n
94     epsilon = np.sqrt(np.log(2 / alpha) / (2 * n))
95     lower = np.clip(fn - epsilon, 0.0, 1.0)
96     upper = np.clip(fn + epsilon, 0.0, 1.0)
97
98     if np.ndim(x) == 0:

```

```

99         return float(fn[0]), float(lower[0]), float(upper[0])
100     return fn, lower, upper
101
102
103 def make_x_grid(sample, kind, frozen_dist):
104     sample = np.asarray(sample)
105     left, right = frozen_dist.support()
106
107     if kind == "continuous":
108         if np.isfinite(left) and np.isfinite(right):
109             span = right - left
110             pad = 0.05 * span if span > 0 else 1.0
111             return np.linspace(left - pad, right + pad, 600)
112
113         q_left = frozen_dist.ppf(0.001)
114         q_right = frozen_dist.ppf(0.999)
115         left = min(sample.min(), q_left)
116         right = max(sample.max(), q_right)
117         pad = 0.05 * (right - left) if right > left else 1.0
118         return np.linspace(left - pad, right + pad, 600)
119
120     left = int(np.floor(left)) - 1
121     right = int(np.ceil(right)) + 1
122     return np.arange(left, right + 1)
123
124
125 def build_demo_table(config):
126     rows = []
127     for item in config:
128         sample = samples[item["sample_name"]]
129         x0 = float(np.median(sample))
130         fn, low, high = empirical_cdf(sample, x0)
131         rows.append(
132             {
133                 "Выборка": item["sample_name"],
134                 "x": round(x0, 4),
135                 "F_n(x)": round(fn, 4),
136                 "Нижняя граница 95% ДИ": round(low, 4),
137                 "Верхняя граница 95% ДИ": round(high, 4),
138             }

```

```

139         )
140         return pd.DataFrame(rows)
141
142
143 def plot_with_custom_ecdf(ax, sample, title, kind,
144     frozen_dist):
145     x_grid = make_x_grid(sample, kind, frozen_dist)
146     true_cdf = frozen_dist.cdf(x_grid)
147     emp_cdf, low, high = empirical_cdf(sample, x_grid)
148
149     ax.plot(x_grid, true_cdf, color="crimson", linewidth=2,
150     label="Истинная CDF")
151     ax.step(x_grid, emp_cdf, where="post", color="royalblue",
152     linewidth=2, label="ЭФР")
153     ax.step(
154         x_grid,
155         low,
156         where="post",
157         color="forestgreen",
158         linestyle="--",
159         linewidth=1.5,
160         label="95% ДИ",
161     )
162     ax.step(
163         x_grid,
164         high,
165         where="post",
166         color="forestgreen",
167         linestyle="--",
168         linewidth=1.5,
169     )
170     ax.set_title(title)
171     ax.set_xlabel("x")
172     ax.set_ylabel("F(x)")
173     ax.set_ylim(-0.05, 1.05)
174
175 def plot_with_scipy_ecdf(ax, sample, title, kind, frozen_dist
176     ):
177     x_grid = make_x_grid(sample, kind, frozen_dist)

```

```

175     true_cdf = frozen_dist.cdf(x_grid)
176
177     ecdf_result = st.ecdf(sample)
178     ecdf_values = ecdf_result.cdf.evaluate(x_grid)
179     ci = ecdf_result.cdf.confidence_interval(confidence_level
180 =0.95)
181     low = ci.low.evaluate(x_grid)
182     high = ci.high.evaluate(x_grid)
183
184     ax.plot(x_grid, true_cdf, color="crimson", linewidth=2,
185 label="Истинная CDF")
186
187     ax.step(
188         x_grid,
189         ecdf_values,
190         where="post",
191         color="royalblue",
192         linewidth=2,
193         label="ЭФР SciPy",
194     )
195
196     ax.step(
197         x_grid,
198         low,
199         where="post",
200         color="forestgreen",
201         linestyle="--",
202         linewidth=1.5,
203         label="95% ДИ",
204     )
205
206     ax.step(
207         x_grid,
208         high,
209         where="post",
210         color="forestgreen",
211         linestyle="--",
212         linewidth=1.5,
213     )
214
215     ax.set_title(title)
216     ax.set_xlabel("x")
217     ax.set_ylabel("F(x)")
218     ax.set_ylim(-0.05, 1.05)

```

```

213
214
215 print(build_demo_table(plot_config))
216
217
218 fig, axes = plt.subplots(4, 2, figsize=(16, 18))
219 axes = axes.ravel()
220
221 for ax, item in zip(axes, plot_config):
222     sample = samples[item["sample_name"]]
223     plot_with_custom_ecdf(ax, sample, item["title"], item["
        kind"], item["dist"])
224
225 handles, labels = axes[0].get_legend_handles_labels()
226 fig.legend(handles, labels, loc="upper center", ncol=3,
        frameon=True)
227 fig.suptitle("Собственная реализация эмпирической функции рас
        пределения", fontsize=16)
228 fig.tight_layout(rect=[0, 0, 1, 0.97])
229 plt.show()
230
231
232 fig, axes = plt.subplots(4, 2, figsize=(16, 18))
233 axes = axes.ravel()
234
235 for ax, item in zip(axes, plot_config):
236     sample = samples[item["sample_name"]]
237     plot_with_scipy_ecdf(ax, sample, item["title"], item["
        kind"], item["dist"])
238
239 handles, labels = axes[0].get_legend_handles_labels()
240 fig.legend(handles, labels, loc="upper center", ncol=3,
        frameon=True)
241 fig.suptitle("Встроенная эмпирическая функция распределения и
        з scipy.stats", fontsize=16)
242 fig.tight_layout(rect=[0, 0, 1, 0.97])
243 plt.show()

```

Листинг 1: Код программы для лабораторной работы №3